

JHQ-GPU: 写作手册

第 3、4、6 节——骨架, 以及每张图连同它的证据

分支 `fix/recall-eval-v15` — 生成于 2026-09-10 — 16 张图

怎么用。第一部分是写作骨架, 每张图放在它所属的章节, 正文和证据在同一页。第二部分是图鉴: 每张图整页宽, 底下跟着它的中文说明——测量的数字、被排除的运行、以及适用的噪声地板都在那里。图注从第二部分写, 不要从第一部分写: 第一部分里那些一行描述是标签, 不是主张。

论文正文用英文写。本中文版是对照参考: 数字、路径、参数名一律保持原样。两版如有出入, 以 `handbook.pdf` 和 `paper_adc2026/README.md`(证据表) 为准。骨架里那节「已被推翻的测量」列出了已经撤回的说法。

重新生成: 先跑 `figures/make_all.sh`, 再跑 `python3 build_handbook.py --zh`。

目录

第一部分——骨架, 图在各自的章节	2
ADC 2026: 第 3、4、6 节母骨架 v3(中文版)	2
已被推翻的测量——动笔前先读	2
论文契约与贡献层级	2
3 GPU-Native JHQ	3
4 自适应层级精化	5
6 实验	6
图与产物完成度对照	15
证据就绪度与剩余实验 TODO	16
第二部分——图鉴	17
fig_pipeline the JHQ-GPU pipeline, shaded by what changed	18
fig_lut the Cartesian factorisation, and what it saves per query	19
fig_layout the transpose, and four subspaces in one load	20
fig_rule how the budget is chosen, and one real calibration	21
fig_frontier six-panel recall-QPS frontier	22
fig_alpha ranking loss vs alpha; what the rule is worth	23
fig_batch throughput and the JHQ/RaBitQ ratio against batch	24
fig_ablation what paid, and what did not	25
fig_calibration the rule against the sweep; sample size; tolerance	26
fig_economics what the calibration costs, and when it is repaid	27
fig_negatives the table-free distance, and reuse as a controlled proxy	28
fig_hierarchy what the second level buys	29
fig_cost JHQ's build, split into training and encoding	30

fig_build	index build time, all five methods	31
fig_lutgroups	why halves; and the table x layout 2x2	32
fig_memory	where JHQ\'s memory goes, and why it is above RaBitQ\'s	33

第一部分——骨架, 图在各自的章节

ADC 2026: 第 3、4、6 节母骨架 v3(中文版)

本文是 ADC_sections_3_4_6_mother_skeleton_v3.md 的中文对照版, 供写作时参考; 论文正文仍用英文写。数字、文件路径、参数名、[TBD] 标记一律保持原样。两版内容如有出入, 以英文版和 README.md(证据表) 为准。

仓库分支 fix/recall-eval-v15。证据审计日期 2026-09-10。下文路径相对仓库根目录; 本文里的 data/ 指 paper_adc2026/data/。版本号 (v47、v52 之类) 只出现在内部证据映射里, 绝不进入论文正文。既有的评审是建议, 冻结结果和实现语义优先。[TBD] 表示该主张或所需验证尚未达到可发表状态。

已被推翻的测量——动笔前先读

早期骨架和评审里有三个数字比证据旧。它们对当时那批运行是真的, 对论文当前数据不是。README.md 是证据表, 且是最新的。

旧说法	现在测到的	出处
「样本量约 S=32」 / 「S=32 是拐点」	拐点是每个工作负载各不相同的。 2000 次重采样、只在留出查询上打 分:S=32 时 openai3-3072 有 76% 落在 1e-3 内, 而 vogue-768 只有 27% , 留出 集平均损失 0.0033、95 分位 0.0110。 openai3-3072 在 S=64 达到 98%;vogue-768 到 S=128 仍有 11% 落 在外面。按声明的风险给 S, 不要给常 数。	data/alpha_resample.json、figures/ alpha_resample.py
「分解 LUT 约 +6% 到 +14.5%」	M=96 时 -4%,M=384 时 +53% 。原来 那 28 个格子全部是 M=96 和 M=128。 256 项表在 M=96 是 98 KiB(共享内存 放得下),M=384 是 393 KiB(放不下): 分 解恰好在完整表放不下的地方才值钱。 M=96 有一格是负的。	data/lut_groups.log、figures/ fig_lutgroups.py
任何说粗量化器欠训练的话	在报告的运行里它不欠训练。_pa.sh 对每个数据集都传 JHQ_N_TRAIN = 39 × nlist。欠训练是历史问题。	SKELETON_REVIEW_v2.md

另有两个结果是骨架当时不可能想到要问、而现在已经测了的:

- 为什么是对半分, 不是四分之一。G=2 在全部八个配置上胜过 G=1、G=4、G=8; 而 G=4 和 G=8 的表更小, 却按每候选每子空间的加载数成比例地更慢。扫描是 issue-bound(受指令发射限制)——这和免表距离、字打包是同一结论的第三个方向。(§6.4.2,fig_lutgroups。)
- 两笔收益相乘。分解在有无 packed 布局时价值几乎相同, 所以「付两次红利」是独立相乘而非重叠——这一点原来的表述是无证据假设的。(fig_lutgroups(b)。)

论文契约与贡献层级

保持老师要求的顺序:1 引言;2 预备;3 方法一;4 方法二;5 相关工作;6 实验;7 结论。详细文献讨论留在第 5 节。

论文故事:

我们利用 JHQ 主表示的笛卡尔结构, 为 GPU 执行重新设计其主距离路径; 并用输出稳定性标定来决定一个工作负载实际需要多少残差精化。

第 3 节问: JHQ 的哪些结构值得在 GPU 上利用。第 4 节问: 该用多少精化, 标定何时回本。第 6 节问: 匹配且可复现的实验是否支持这两个答案。这是表示、执行、策略、证据四件事, 不是 CUDA 改动的编年史。

组件	科学地位	要求的处理方式
笛卡尔分解主 LUT 主码打包加载	最强的 JHQ 专有贡献 表示使能的优化	推导精确恒等式, 并测量其执行收益。 讲清可容许性与指令数下降; 地位次于代数贡献。
子空间主序物理布局 批量 GEMM / 正交变换	必要的标准 GPU 工程 使能基础设施	讲合并访存, 但不要把转置说成新颖性。 保持数学语义不变。
按查询数发射、游标遍历删除、bug 修复	实现修正	与研究贡献、与代数消融分开。
输出稳定性 alpha 选择	策略/算法贡献	选择质量与经济性分别验证。

新颖性和效应量是两个轴: 打包的实测加速可能比分解大。如实报告两者, 不要为了迁就性能而改动新颖性标签。不要断言「没有别的量化器能做分解」; 精确分解来自这一笛卡尔表示, 而对比的基线在其当前表示下不具备这一性质。基线表示的具体引用留 [TBD], 文献阶段补。

3 GPU-Native JHQ

3.1 设计总览

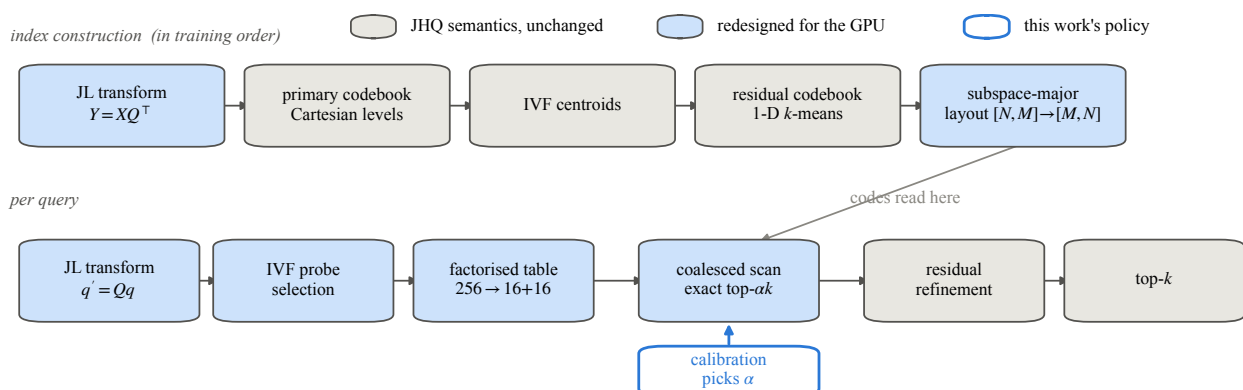


图 1: **fig_pipeline**. the JHQ-GPU pipeline, shaded by what changed 完整说明与全部注意事项见第二部分 `fig_pipeline.py`。

用 `fig_pipeline` 介绍两级表示和 GPU 查询路径。主扫描对被路由到的候选打分、保留主幸存者, 只对这些幸存者用残差信息精化, 再做最终 top-k 选择。表示决定哪些距离计算可以被精确重组; 精化预算决定近似过滤发生多少。

记号统一: 库向量 x , 变换后 $y = Qx$, 变换后查询 $q' = Qq$, $Q \in \mathbb{R}^{(d \times d)}$, M 个子空间, 主码 c_m , 请求输出规模 k 。JL/正交变换是方阵, 不是降维。批量按行时 $Y = XQ^T$ 是同一约定。

3.2 面向 GPU 的表示与布局

把批量变换、主码与残差编码、IVF 组织、物理布局连成一段来讲。残差是 $r = y - \hat{y}_{\text{primary}}$, 绝不是 IVF 质心残差。IVF 组织候选路由, 它不重新定义残差目标。查询侧 ADC 用的是由完整查询构建的表, 不是单个量化后的库式查询码。

描述逻辑上的候选主码矩阵 $[N,M]$ 与扫描用的子空间主序布局 $[M,N]$, 然后是打包的物理变体。逻辑布局 and 打包存储要分清。批量 GEMM 和转置是实现基础设施, 不是独立的研究主张。

`fig_pipeline` 里的训练依赖必须和代码一致: 变换与主码构建先于 IVF 质心训练和残差码本训练; 残差码本用采样, 不必等所有库向量分配完毕。不要硬编码公式编号, 也不要画出暗示错误依赖的示意。

3.3 结构感知的查询处理

3.3.1 笛卡尔分解的主距离表 每个子空间 $T_m[c] = \|q'_m - C_m[c]\|^2$ 。在当前的笛卡尔 8 位表示下, 把坐标和码分成互不相交的高低两半。平方欧氏距离在互不相交的坐标上可加, 于是精确地有:

$$T_m[c] = T_m^{hi}[c \gg 4] + T_m^{lo}[c \bmod 16]$$

于是每子空间 256 项变成 $16 + 16 = 32$ 项。这些计数是代数推论, 不是实测加速。讲清这条链: 笛卡尔结构 → 精确分解 → 更小的查询专属表和更少的构建工作 → 再评估 GPU 吞吐。多出来的查表和算术意味着表变小不等于吞吐按比例变好。

`fig_lut` 放这里。实现会对着质心检查笛卡尔可分性; 不能悄悄换成一般的细化码本。代数精确不等于浮点逐位相同, 也不等于并列顺序确定。

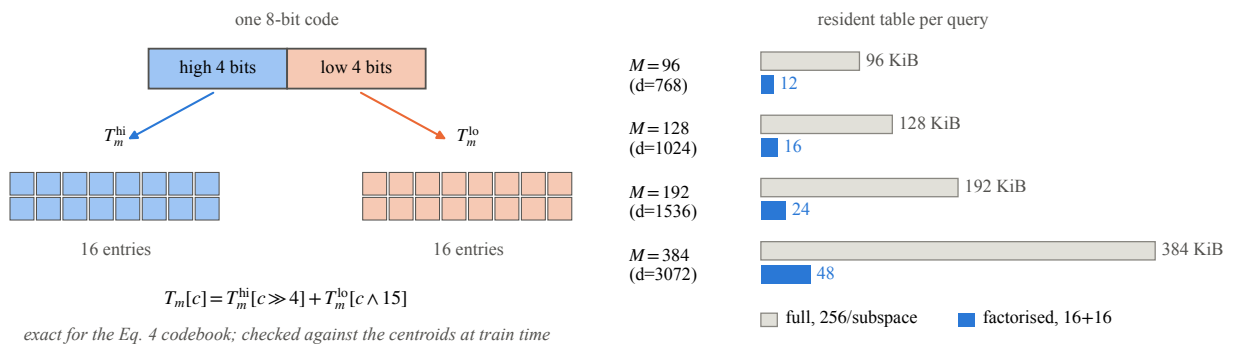


图 2: **fig_lut**. the Cartesian factorisation, and what it saves per query 完整说明与全部注意事项见第二部分 `fig_lut.py`。

3.3.2 合并访存的主扫描与打包访问 讲固定子空间上的连续候选访问、主距离累加、幸存者选择。当前一字节的主码允许把四个子空间打包进一个 32 位字。把打包呈现为「由这一表示使能」以及「减少加载/寻址/循环指令」。在字节布局本来就已合并的情况下, 不要暗示打包必然减少传输字节。不要重复那个错误的 128 字节行 / 浪费 75% 的解释。

`fig_layout` 把布局的合并效应和打包的指令效应分开。按查询数发射、冗余游标删除属于实现说明和一个单独标注为「修正」的消融。

3.3.3 选择性残差精化 讲主幸存者的残差解码/打分与最终 top-k 选择。要区分「对一个选定的近似表示做精确求值」和「对原始向量做精确最近邻搜索」: 路由和主过滤仍可能排除真近邻。用 §6.4 的仅主码消融来给层级做动机, 但不要暗示穷举路由能弥补不足的主码分辨率。

3.4 剩下的策略问题

执行路径接受一个幸存者预算, 但不决定这个预算该多大。** 明确以 $ck = \alpha \times k$ 和这个问题收尾:**

到底该精化多少个主幸存者?

4 自适应层级精化

4.1 动机与工作负载依赖

原版 JHQ 把 α 留给外部指定。精化太少会丢近邻; 太多则返回同样的 top-k 却白花残差计算。

用这个有界的经验陈述: 在 $nprobe=128$ 下, 足够的精化预算从 4 到至少 200; 上端是下界, 因为实测曲线在最大测试 α 处仍在改善。证据用 `data/alpha6.log` 的 $Br=8$ 行, ** 不要用 `alpha_ds.log`, 后者没测 200。OpenAI-3072 在 $\alpha=4$ 和 200 的召回都是 0.9416; arxiv 从 100 的 0.9760 升到 200 的 0.9771。这是一个受网格限制的操作性饱和观察, 不是精确比值, 也不是网格之外存在平台的证明。不要用「25×」这个说法。 **

4.2 输出稳定性判据

设 Q_s 是 S 条抽样查询, $R_{\max}(q)$ 是在充裕的 $\alpha_{\max}$ 下的 top-k ID 集合。同样定义 $R_{\alpha}(q)$, 并把 ϵ 定义为整个样本上不一致槽位的非负整数计数:

$$D(\alpha) = \sum_{\{q \in Q_s\}} [k - |R_{\alpha}(q) \cap R_{\max}(q)|] \quad \alpha^* = \min \{ \alpha \in \text{测试网格} : D(\alpha) \leq \epsilon \}$$

这是每查询的集合重叠: 顺序不同但集合相同算一致。它既不是分数容忍度, 也不是每查询的 ϵ , 也不是位置排名的不一致。它假定 top-k ID 有效且互异。当前默认 $S=32$ 、 $\epsilon=1$; 这些是经验设定, 不是正确性保证。

规则不用任何外部 ground truth 标签, 也不用学习到的预测器。ground truth 只在事后用来评估策略。与 $\alpha_{\max}$ 一致并不能证明精确近邻召回、不能挽回 IVF 路由漏掉的、也发现不了 $\alpha_{\max}$ 之外的改善。

4.3 选择过程与定位

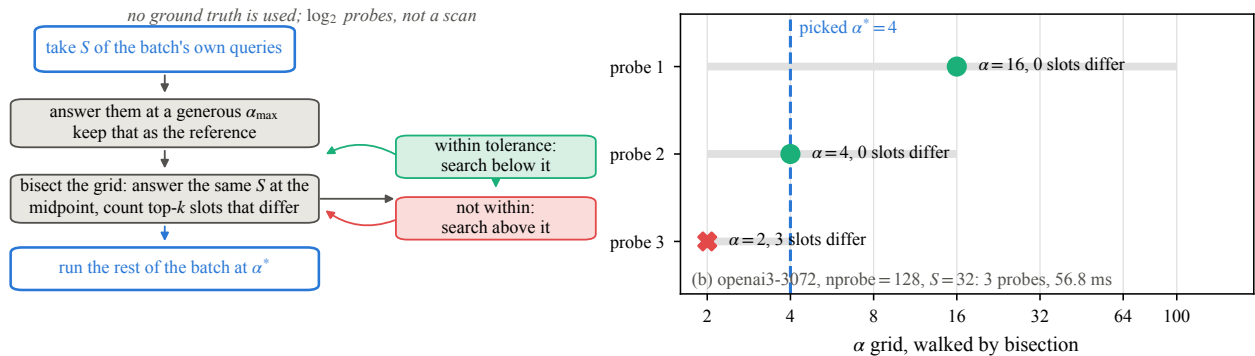


图 3: **fig_rule**. how the budget is chosen, and one real calibration 完整说明与全部注意事项见第二部分 `fig_rule.py`。

算法框: 选样本和降序 α 网格; 算出 $\alpha_{\max}$ 参考; 用 D 评估更小的预算; 选中一个被接受的预算; 把它用于后续工作负载查询。 ** 把实际的搜索调度、样本选取方式和停止条件与 `fig_rule` 一并报告。 **

内部实现证据: `examples/demo_jhq_alpha_fast.cu` 默认网格 $\{100, 64, 32, 16, 8, 4, 2\}$, 从当前批次里按等间隔选查询, 默认走二分。它另有一个可选的线性模式, 逐级下降并在第一次拒绝时停止。上面那个概念上的最小值是目标; 要断言实现总能找到全局最小, 需要「被接受前缀/单调性」假设。并列行为和随预算变化的选择需要验证, 不能给无条件保证。用这句话: 「停止行为相对于被采样的一致性判据是保守的。」不要说它绝不会选得太小。

写作用的定位段落: 基于采样的参数调优和自适应 ANN 搜索控制已经存在, 包括 FAISS ParameterSpace 和自适应提前终止方法。我们的区别在于: 无标签、无预测器的 top-k 输出稳定性, 直接作用于 JHQ 外部指定的 α , 并且对着它意图取代的那个穷举扫描做了验证。不要声称采样本身新颖, 也不要声称全网格验证已经完成。主要文献引用和精确比较留 [TBD], 第 5 节定稿。

4.4 标定成本与复用

记 T_{cal} 为标定时间, T_{fixed} 和 T_{rule} 为固定 α 与选定 α 下每个同等大小后续批次的时间, B 为后续批次数:

$$T_{adaptive}(B) = T_{cal} + B \cdot T_{rule} \quad T_{fixed_total}(B) = B \cdot T_{fixed} \quad B^* = T_{cal} / (T_{fixed} - T_{rule}), \text{ 当 } T_{rule} < T_{fixed} \text{ 时}$$

批次是整数, 取满足所需非负/严格节省条件的最小整数。若 $T_{rule} \geq T_{fixed}$, 则没有有限回本。分数形式的 B^* 描述的是摊销, 不是一个分数的实测工作负载。这个比较以「观察到的质量差可接受」为前提; 原始增益不自动等于等召回增益。

复用假定查询分布、路由配置、 k 和索引都稳定。漂移和重标定频率是未测量的局限。每 H 批重标定一次, 在此模型下每批增加 T_{cal}/H ; 不要因为只采样了一小部分查询就推出某个固定百分比的开销。当前标定计时从样本拷贝之后开始, 不含该计时器之外的准备工作; 要把日志里的标定时间和未来一个全包含的部署开销测量区分开。

6 实验

6.1 实验设置与协议

正文保持简洁, 但下列每条协议原则都要覆盖, 完整网格和来源放到 artifact 表格/附录。缺失项保持 [TBD], 不要从另一批实验借用假设。

- **硬件与数据:** 冻结项目用一块 RTX 5090, 标称 32 GB。设备可用显存的读数当作测量值报告, 不要当规格。覆盖 vogue-768、arxiv-768、bge-m3、stella-trec24、openai3-1536、openai3-3072。精确的 N 、预处理、查询数、度量与 GT 来源须与基准输入对齐 [TBD]; 数据集展示名里的 N 可以取整。
- **Recall@10**, 当前评估全程 $k=10$ 。 α 与 k 是耦合的; 跨 k 的推广性未测量。查询 ID、GT、距离约定和计时边界在各系统之间共享。统一的计时边界是「主机查询进 → 主机结果出」; 每个可执行文件的同步、预热、计时重复次数都要记录。规则的基准用一次预热加三次完整搜索计时 (examples/demo_jhq_alpha_fast.cu); 这不构成每个基线的协议。
- **Recall-QPS** 曲线报告稳态查询吞吐; 标定成本单独报告, 并通过回本分析纳入。 AF_RESULT 把标定和整批吞吐分开计时。不要把诊断用的 α 扫描 QPS 放进前沿图——它的诊断回读会改变计时。
- 必须有的 **CPU JHQ**: 最强的、可辩护的、可复现配置, 并给出源码版本、线程数、亲和性/pinning、训练预算、等召回和计时。要尝试原作者的 artifact; 如果手上的基线是 JHQ_repro, 要标明它是重实现而非作者的可执行文件。冻结的 artifact 说明记录了构建不兼容和未匹配的残差训练。历史上 32 线程胜过 208 线程是一个协议警告, 不是已定稿的 GPU 加速比。CPU 加速比 [TBD], 直到用匹配训练和可辩护的 pinned 配置重跑为止。若最终无解, 必须明确披露缺失。
- 主要 **GPU 基线**: IVF-RaBitQ、CAGRA、IVF-PQ。可选参考: IVF-Flat、暴力搜索、仅主码/类 JQ 的层级消融。记录库版本、构建与搜索网格、nlist、nprobe、PQ 码长、RaBitQ 模式、CAGRA 图/构建/搜索参数与精度。要么说明显存/码长预算已匹配, 要么明确承认预算不同。最终基线网格/来源表 [TBD]。
- 当前 JHQ 的 FIX/RULE 网格在 data/paper_fronts.log: nprobe={8,32,128,256,512,1024}; (M,nlist) 为 vogue (96,4096)、arxiv (96,8192)、BGE (128,32768)、stella (128,32768)、OpenAI-1536 (192,8192)、OpenAI-3072 (384,4096)。这些是观察到的设定, 不是穷尽调参的证明。保留 QUANT4 的证据及其冻结出处, 不要悄悄换成更慢的 RaBitQ 模式。
- 前沿点的选取遵循 figures/style.py 里的非支配包络。给定召回下的 QPS 在相邻保留测量点之间做对数线性插值; 由此得到的比值要标明是插值, 且绝不外推到任一方法的实测范围之外。公布精确的源数据点和参

数配置。一个扫描端点不是架构上的召回上限。

- 披露粗训练历史: 早期配置每质心约 6 个训练点, 后来用了密得多的训练, 约 39。results/front6/README.md 与 NTRAIN.md 区分了这两种训练制度。为每一批被绘制的数据冻结确切的训练配方/缓存来源 [TBD]; 绝不把修复前和修复后的 JHQ 点混成一条最终前沿。
- 披露 **Vogue** 的不平衡粗桶, 把它当作所观察到的已训练索引的一个特征, 不要指派未经证实的原因, 也不要用它开脱差的表现。README 报告 135,489/932,328 向量, 约 14.5%, 但它引用的 data/export_ivf.log 不在当前冻结目录里。** 发表这个百分比之前先冻结原始直方图证据 [TBD]。 **
- 可重复性: results/front6/README.md 报告冷缓存下 Vogue 召回 0.9852/0.9842/0.9849, 候选集相同。除非有重复证据澄清, 低于约 $1e-3$ 的变化视为处在观察到的 **build** 间波动内。这是一个观察到的范围, 不是置信区间, 也不是普适的噪声阈值。原始重复运行证据和系统性计时不确定度仍 [TBD]。

6.2 端到端 Recall-QPS ——RQ1

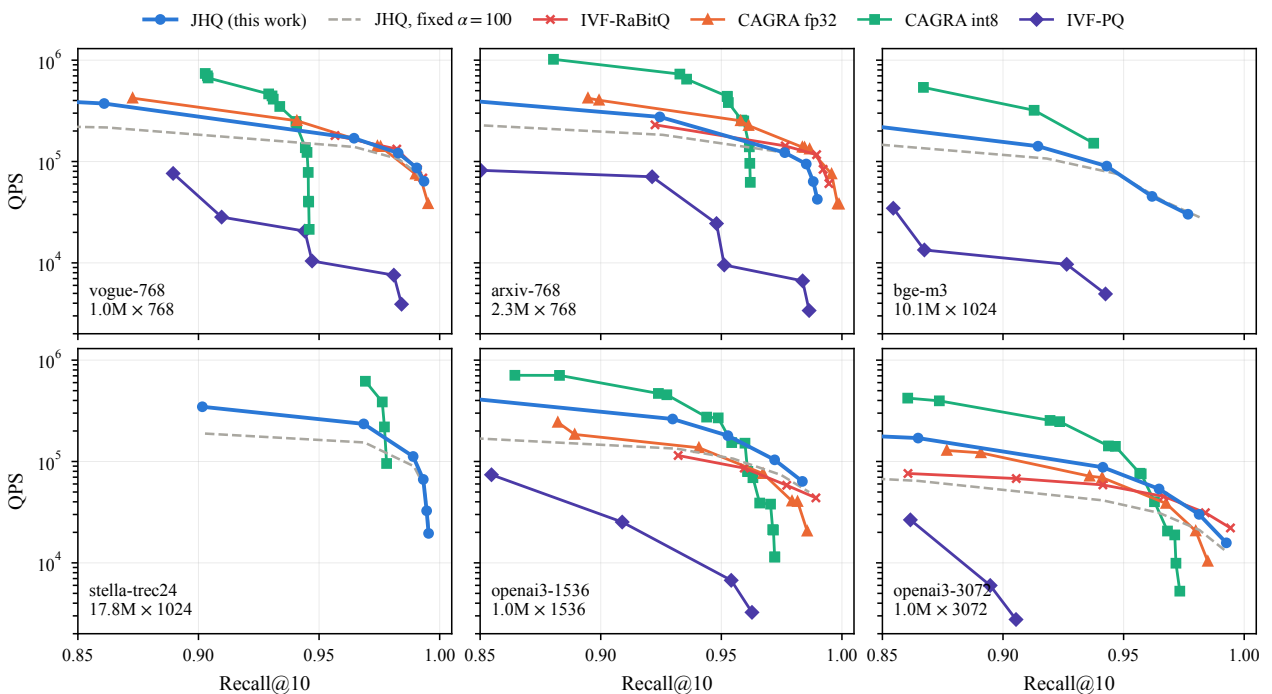


图 4: **fig_frontier**. six-panel recall-QPS frontier 完整说明与全部注意事项见第二部分 `fig_frontier.py`。

用 `fig_frontier` 在公共召回区间上比较各方法的工作区域。证据来源: `data/paper_fronts.log`, `data/paper_rabitq.log`, `data/bench_quant.log`, 以及 `**report_adc2026/v47/fronts.json`。最后一个提供的是较早的 **CAGRA/IVF-PQ** 扫描, 不是当前的 JHQ 点 **。把合成图当作定稿比较之前, 先核实协议兼容性。

按数据集分区域来写: JHQ 在中高召回可以很强, 尤其在更高维的 OpenAI 工作负载上; RaBitQ 在高召回尾部可能追上或反超。只量化那些有曲线重叠支撑的插值等召回比较。没有普适赢家, 也不能仅凭维度推断因果。

CPU JHQ 应进入一张紧凑的等召回表格并附来源与协议, 不是可选的脚注; 当前条目 [TBD]。CPU 吞吐要与 GPU 前沿的坐标轴分开。分配失败必须指明所测的配置和库路径; `data/rabitq_bigsets.log` 与 `data/paper_rabitq.log` 支持的是有限的失败披露, 不是「该算法永远无法索引这些数据集」。CAGRA 的各精度变体必须分别标识。

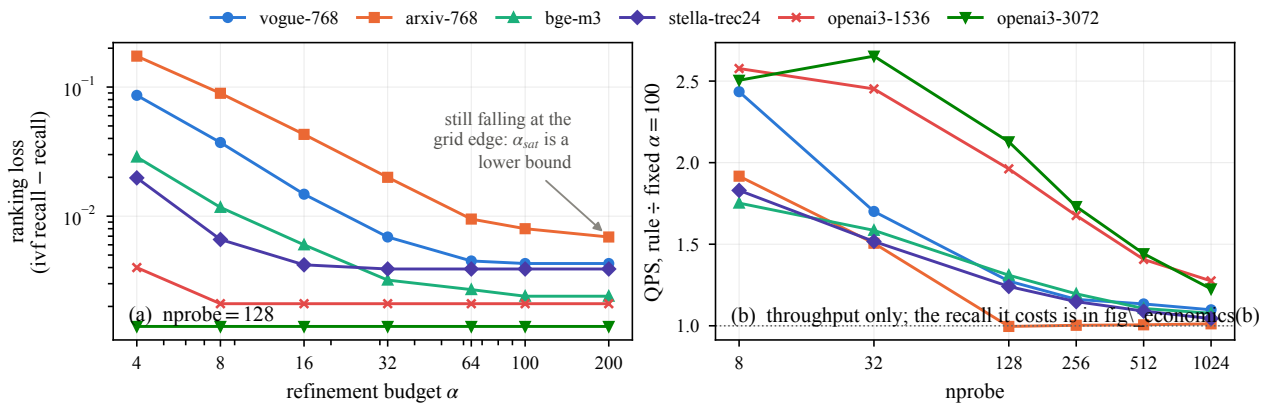


图 5: **fig_alpha**. ranking loss vs alpha; what the rule is worth 完整说明与全部注意事项见第二部分 *fig_alpha.py*。

6.3 自适应精化评估——RQ2

6.3.1 足够 α 的变化幅度 用 *fig_alpha* 和 $\text{nprobe}=128$ 下 $\text{Br}=8$ 的 $\alpha_{\text{br}}.log$ 扫描。展示网格和 arxiv 右删失的端点; 用「4 到至少 200」。当前绘制的量是 $\text{ivf_recall} - \text{recall}$, 所以在明确给出这个定义的前提下, 「ranking loss(排序损失)」是恰当的: 路由损失已被剔除。若改画 $1 - \text{Recall}@10$, 则应标为 recall loss/error。不要拿诊断 QPS 当性能证据。

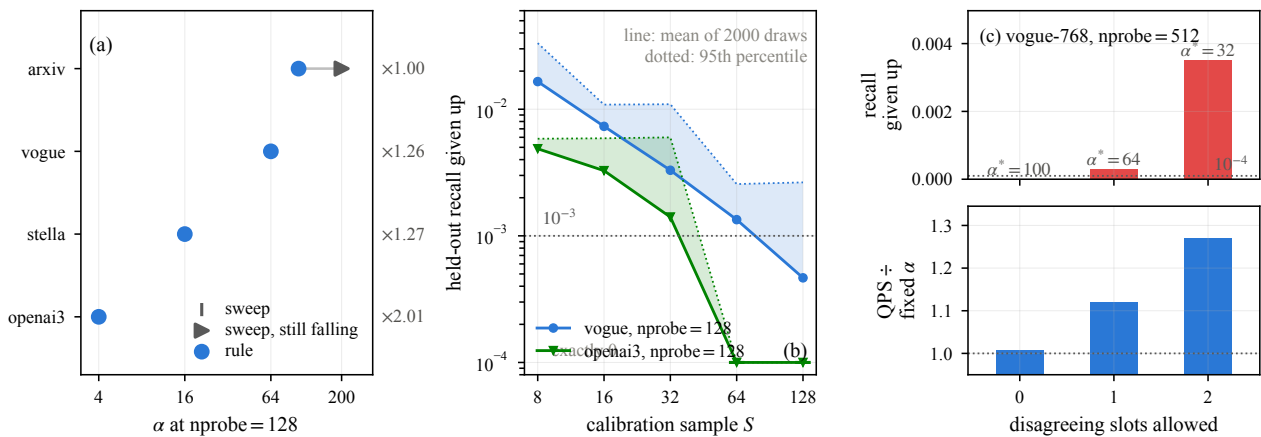


图 6: **fig_calibration**. the rule against the sweep; sample size; tolerance 完整说明与全部注意事项见第二部分 *fig_calibration.py*。

6.3.2 标定 vs 穷举扫描 用 *fig_calibration*, 参考值按 (**dataset**, **nprobe**) 键控。当前面板 (a) 比较 $\text{nprobe}=128$ 上的四个采样规则配置, $\text{nprobe}=512$ 没有对应的穷举扫描, 不得复用另一个 **nprobe** 的答案。脚本用相对最大网格排序损失 $3e-4$ 的容差来推导扫描 α 。那个操作阈值小于观察到的 **build** 间范围, 不要把它说成已确立的噪声地板。对平台定义的敏感性需重估 [TBD]。

data/alpha_sample.log 支持样本量敏感性, 包括 OpenAI-3072 在 $S=8$ 选中 $\alpha=2$ 并损失 0.0058 召回。但那个较早的实现用的是分数容忍度, 它不是最终整数槽位规则的完整敏感性研究。*data/alpha_fast.log* 支持 $\text{epsilon}=\{0,1,2\}$ 的槽位实验; 要区分线性行和二分行, 不要让解析器把重复配置互相覆盖。Vogue 在 $\text{nprobe}=512$ 时, 两槽二分之一选中 32 并损失 0.0035 召回。不要把一个数据集的容忍度选择推广到所有工作负载。

最终验证 [TBD]: 用最终策略重跑样本量与槽位敏感性; 在每个测试过的 (**dataset**, **nprobe**) 上, 把选中的 α 和整批召回与穷举网格对比; 记录 α_{max} 的删失; 重复采样/重复 build; 检查一致性的单调性和并列行为。把选择准确性、速度、参考质量三件事分开。

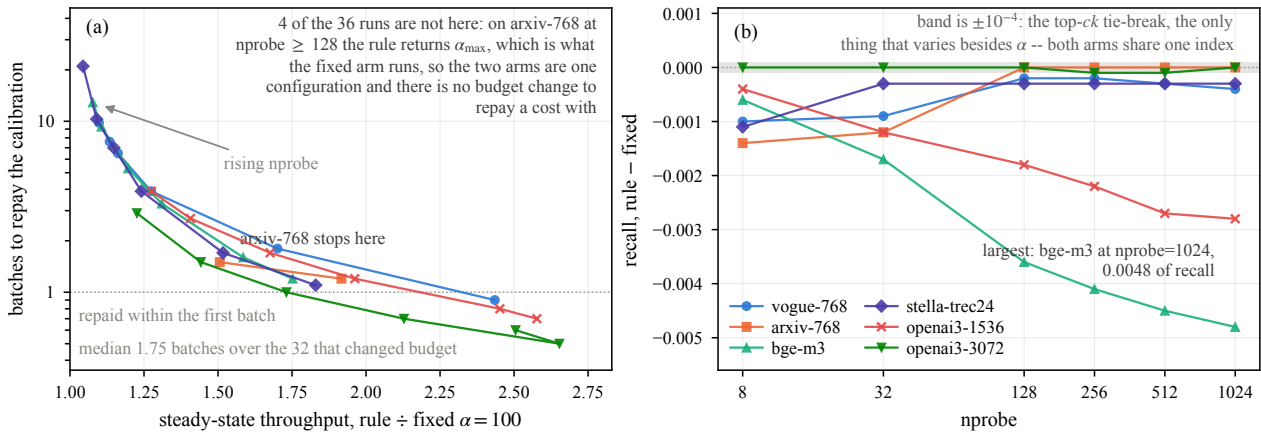


图 7: **fig_economics**. what the calibration costs, and when it is repaid 完整说明与全部注意事项见第二部分 *fig_economics.py*。

6.3.3 稳态增益与回本 把经济性做成头条结果,用全部 RULE 行及其行内配对的 qps_max, 不要用无关的 FIX 计时。包含: 选中的 alpha、稳态增益、召回差、标定毫秒数、batches_to_repay。下表直接由冻结的 data/paper_fronts.log 重算:

量	核实后的值与解读
RULE 配置数	36
日志中的标定时间	2.0–47.5 ms
有限回本范围	0.5–178.6 批
有限回本中位数	35 个有限回上为 2.7 批; 排除 -1 哨兵值
需要两批以上的行	36 个配置中的 18 个; 另有一行永不摊销
无有限回本	arxiv,nprobe=128:gain=0.996, 哨兵 = -1.0
最大原始增益	2.653, OpenAI-3072, nprobe=32; 日志回本 = 0.5
最长有限回本	arxiv,nprobe=256:gain=1.003, 回本 = 178.6
最大日志召回损失	BGE,nprobe=1024:0.0048

**** 本节已被后续测量修正, 见 figures/fig_economics.py 与 SKELETON_REVIEW_v3.md. arxiv 在 nprobe=128/256/512/1024 上选中的都是 $\alpha=100$, 与固定臂完全相同 **;** 两条臂跑的是同一个配置, 所以那几行的 gain 是重复计时波动 ($\pm 1.2\%$), batches_to_repay 是拿标定成本除以噪声, 178.6 是 1/噪声而不是经济学结果。剔除这四行后, 其余 32 个真正改变了预算的配置增益全部 $> 1(1.044-2.653\times)$, 回本 0.5–21.0 批、中位数 1.75。

BGE 的损失在 nprobe=128(0.0036)、256(0.0041)、512(0.0045) 也超过 0.003。因此不要重复 README 那句「除一个外都 ≤ 0.003 」, 也不要把所有原始增益标成等召回增益。像 -0.0003 这样的亚 $1e-3$ 差值处在观察到的 build 间波动内; 更大的损失仍然可见。

fig_economics 已完成: x = 稳态 QPS 增益, y = 有限的 batches_to_repay(对数轴), 按数据集着色/标记。有 gain=1 和 payback=1 参考线。不回本的配置单独用注释表示, 绝不把 -1 画到对数轴上, 也不悄悄丢掉。36 个配置全部有交代。图注要说明 32 行的中位数、取整方式、质量差和稳定工作负载假设。

联合解读: 显著增益倾向于快速回本; 可忽略的增益导致很长或没有回本。有限的 178.6 是「很长」而不是数学意义上的「永不」, 而且微小的计时差需要重复测量。「八批 / 8000 条查询」那个旧说法不是当前结果。经济性衡量的是有条件的工作负载复用, 不是对未来漂移批次的泛化。

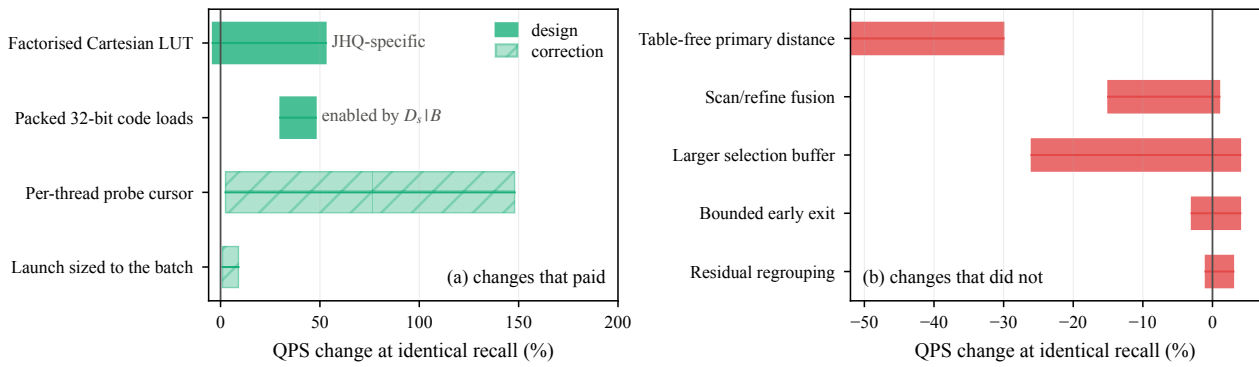


图 8: **fig_ablation**. what paid, and what did not 完整说明与全部注意事项见第二部分 *fig_ablation.py*。

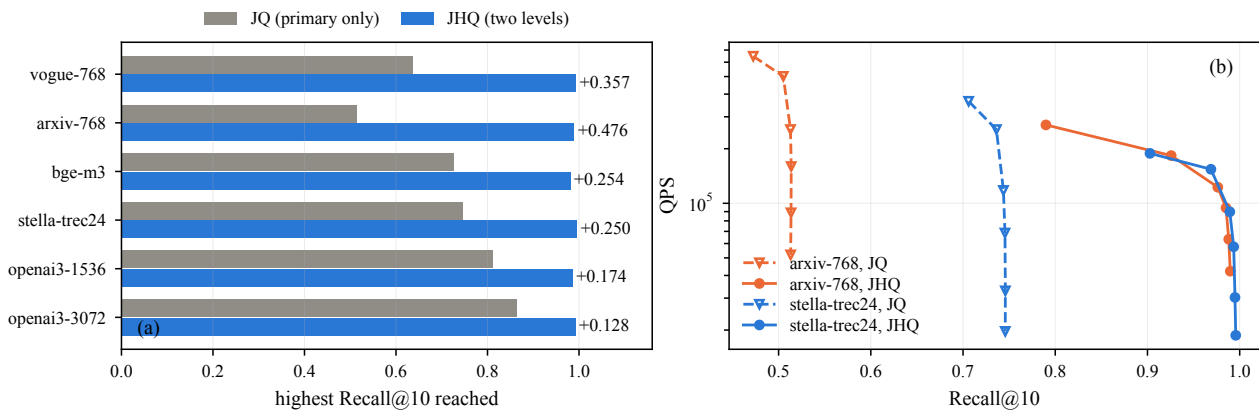


图 9: **fig_hierarchy**. what the second level buys 完整说明与全部注意事项见第二部分 *fig_hierarchy.py*。

6.4 表示与执行消融——RQ3

6.4.1 残差层级的价值 用 *fig_hierarchy*、*data/hierarchy_ablation.log* 和完整的 JHQ 冻结前沿。在匹配的路由/训练下比较仅主码/类 JQ 与完整 JHQ。报告整个扫描上达到的召回, 以及精化增加的工作量。把最大值称为「实测最高召回」, 不要称为架构上限。这个消融测的是层级, 不是一个调优完备的独立 JQ 系统。

6.4.2 笛卡尔分解 LUT 在码、候选、策略完全相同的条件下, 隔离完整表 vs 分解表。先陈述表示的精确等价, 再给实测的构建/搜索效果。历史证据在 *results/v47_split_lut/*。不要用代数上的条目比来冒充加速比。

** 现已完成, 见 *figures/fig_lutgroups.py* 与 *data/lut_groups.log*。61 次运行零失败, 四种粒度在每个配置下召回完全一致 (恒等式要求如此, 这是先于任何计时的正确性检查)。两个结果:(a) 为什么是对半分—— $G=2$ 在全部八个配置上胜出, 而 $G=4$ 、 $G=8$ 的表更小却按加载数 (2/4/8) 成比例地更慢, 说明扫描是 **issue-bound**;(b) M 依赖性——分解在 $M=96$ 值 $-4\%\sim+12\%$, 在 $M=384$ 值 $+23\%\sim+53\%$, 因为 256 项表在 $M=384$ 是 393 KiB、放不进共享内存。分解恰好在完整表放不下的地方才值钱。另外, 分解的收益在有无 **packed** 布局下几乎相同, 所以两笔收益相乘而非重叠 **。

6.4.3 表示使能的优化与工程 按地位区分打包访问、物理布局、游标/发射修正。***fig_ablation* 不应把它们画得同等新颖。** 历史打包与修正证据在 *results/front6/NEGATIVES.md*、*results/front6/v52.log*、*results/front6/v51.log*、*data/v57_launch.log*; 最终匹配消融矩阵见 §6.4.2 的 2×2 。不要把来自不同批次的百分比加起来推断一个合成加速比。即使更简单的优化效应更大, 也要把实测效应量并排如实报告。

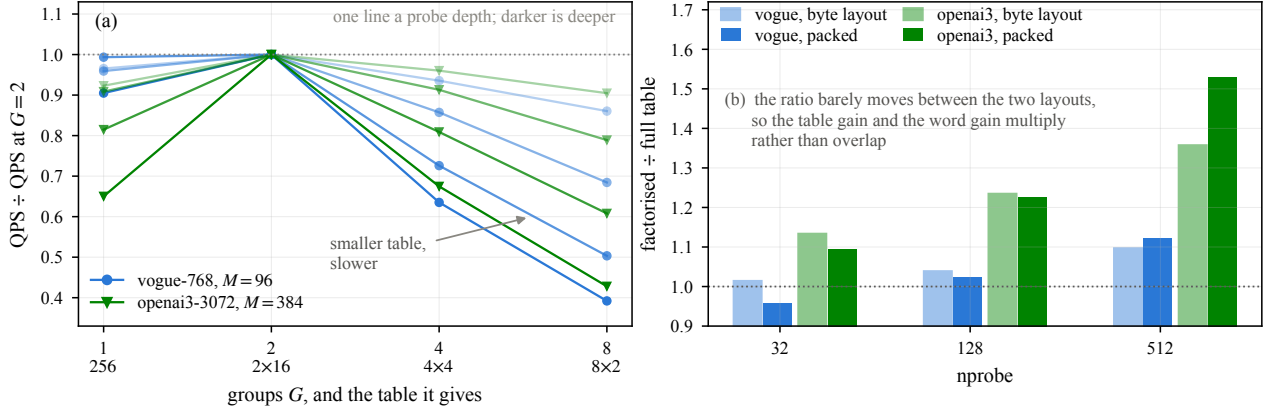


图 10: **fig_lutgroups**. why halves; and the table x layout 2x2 完整说明与全部注意事项见第二部分 *fig_lutgroups.py*。

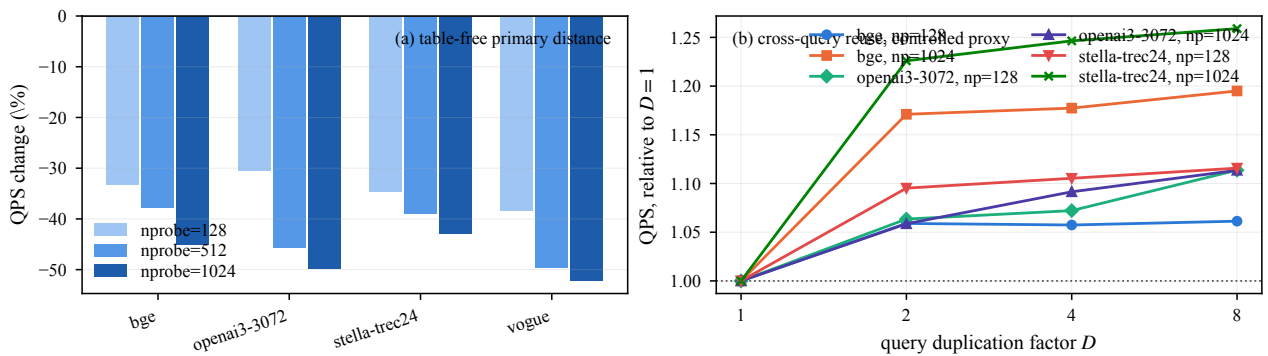


图 11: **fig_negatives**. the table-free distance, and reuse as a controlled proxy 完整说明与全部注意事项见第二部分 *fig_negatives.py*。

6.5 机制分析与负面结果——RQ4

用 `fig_negatives`; 讨论更大的选择缓冲、重分组、扫描/精化融合、提前退出、免表距离。证据: `data/v54.log`、`data/qdup.log`、`data/qdup_stella.log`, 内部实验说明在 `results/front6/NEGATIVES.md`、`V54_SIGN_IP.md`、`QDUP.md`。引用其他负面结果的数值范围前先冻结原始数据 [TBD]。

免表实验说明表的字节更少不等于执行更快。指令数、占用率、缓存、共享内存驻留、访存流量都起作用。把实测的运行时/资源计数与需要 **profiling** 才能支撑的因果解释分开。单卡观察不构成跨架构定律。

重复查询实验是受控的复用代理, 不是对另一种调度器的上界。若 L2 复用是推断出来的, 就说这个模式「与 L2 已经吃掉大部分复用相符」。直接归因和选择阶段的 **profiling** 仍 [TBD]。宁可压缩 **setup**, 也不要删掉这块证据; 真要压缩, 把负面结果并入消融讨论, 但不要把实验组织退化成版本史。

6.6 批大小敏感性——RQ5

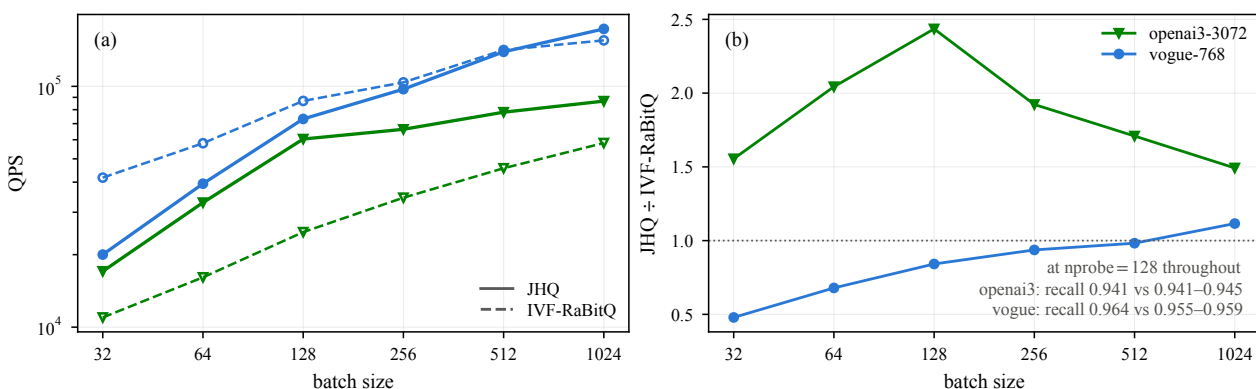


图 12: **fig_batch**. throughput and the JHQ/RaBitQ ratio against batch 完整说明与全部注意事项见第二部分 `fig_batch.py`。

用 `fig_batch` 和 `data/batch_sweep.log`, 明确标注为固定 **nprobe=128** 下的批大小敏感性。日志扫描覆盖 OpenAI-3072 和 Vogue, 批标签从 32 到 1024。把两边的召回和吞吐比一起展示: 在最大批上, OpenAI-3072 的召回是 JHQ/RaBitQ = 0.9414/0.9436, Vogue 是 0.9645/0.9586。这些不是等召回的胜负比较。

在把该工作负载描述成 1024 条独立查询之前, 先核对日志里的 `batch=1024` 与查询文件实际行数及基准的批处理方式 [TBD]。不要靠复制约 1000 条真实查询来制造一个「独立的 10k 工作负载」。等召回的批扫描、独立的更大查询集、批效应的因果归因仍 [TBD]。不要把曲线外推到别的 GPU 或别的批大小。

6.7 构建时间与显存——RQ6

用 `fig_build`、`fig_cost` 和 `fig_memory`。data/vram.log 测了 RaBitQ 在四个数据集上的常驻显存; data/paper_fronts.log 测 JHQ。说明测量点 (索引加已分配的搜索工作区)、单位和 **nprobe**/工作区配置。例如 **nprobe=8** 时 JHQ/RaBitQ 实测值: Vogue 1405.2/1012.0 MiB, OpenAI-3072 4047.2/3324.0 MiB。JHQ 的实测常驻量随 **nprobe** 变化, 当前图的加载器保留最后一条 FIX 行, 所以不要把它柱子和取自第一行的正文数字混用。

把主码/残差码、ID/校正项、质心、查询/候选工作区分别建模, 与实测总量分开。加一段「**other / allocator / runtime workspace** = 实测 - 建模」。这一残差段让账目平衡, 但并不独立测量其成因; 出现负的缺口说明模型有错。竞品的组件只有在有实测的地方报告。RaBitQ 在四个有实测的数据集上常驻占用更小。说「JHQ 处在一个不同的显存-精度-吞吐工作区」, 不要说普遍的显存优势。

构建时间要把训练与编码分开、冷训练与缓存加载分开、输入加载与索引构建分开。



图 13: **fig_cost**. JHQ's build, split into training and encoding 完整说明与全部注意事项见第二部分 *fig_cost.py*。

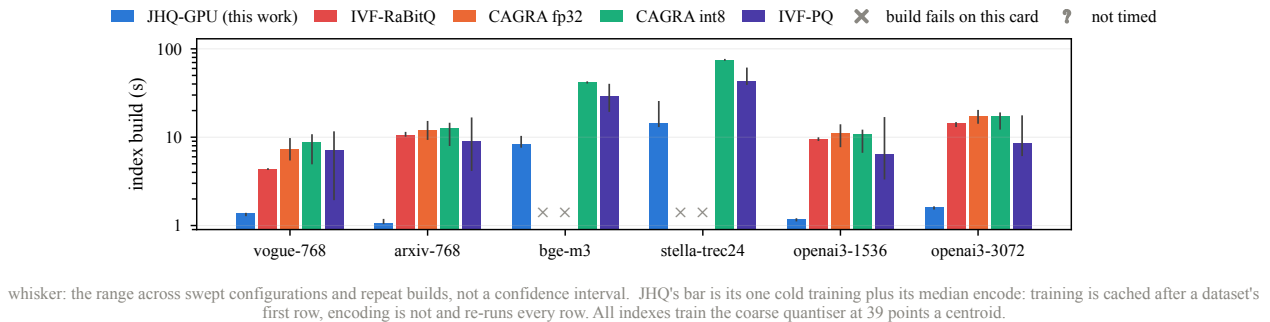


图 14: **fig_build**. index build time, all five methods 完整说明与全部注意事项见第二部分 *fig_build.py*。

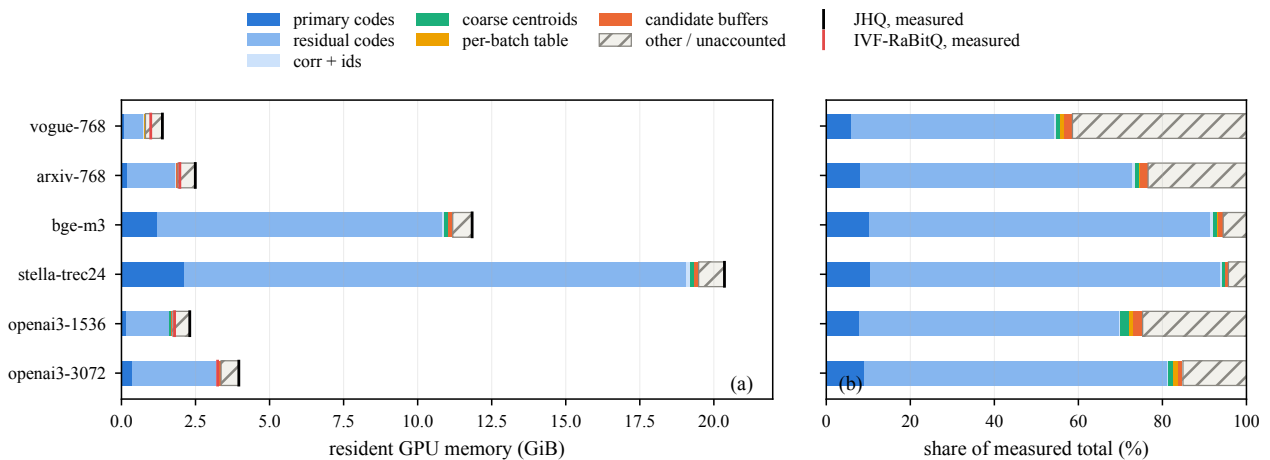


图 15: **fig_memory**. where JHQ's memory goes, and why it is above RaBitQ's 完整说明与全部注意事项见第二部分 *fig_memory.py*。

注意一个已经踩过的坑: 两个构建阶段的缓存行为不同。train 有缓存, 只有每个数据集的第一行 (nprobe=8) 是冷的, 其余报 213–273 ms; add 没有缓存, 每行都完整重跑, 在 stella 上六次从 9.1 s 到 21.8 s。取 $\max(\text{train}+\text{add})$ 会选中「缓存过的训练 + 一次特别慢的编码」, 把一个 13.4 s 的构建报成 22 s。正确做法: 训练取冷的那一行, 编码的散布当作散布报告。

fig_build 给出五个方法的跨方法比较; fig_cost 给出 JHQ 自己的训练/编码拆分。冷缓存来源、单位和竞品等价的计时边界需最终核实 [TBD]。不要拿后面那些命中缓存的训练行、或未匹配的 CPU 训练, 当作完整的构建比较。

6.8 实验结论

只写最终证据支持得住的: 笛卡尔结构使主距离求值可被精确重组; 精化有价值但其有用预算取决于工作负载; 输出稳定性选择必须同时用质量和回本来评判; 有竞争力的吞吐占据特定的数据集/召回/批大小区域, 并伴有明确的显存与构建代价。**CPU 加速主张在必需的基线落地之前保持 [TBD]。**

图与产物完成度对照

图	位置	剩余检查或动作
fig_pipeline	3.1	记号统一用 Q、真实训练依赖、无公式编号。已完成。
fig_layout	3.2 / 3.3.2	把合并访存与打包的指令效应分开; 去掉 cache-line 浪费的谬说。已完成。
fig_lut	3.3.1	用当前的精确表示; 替换硬编码的公式引用。已完成。
fig_rule	4.3	对齐集合槽位判据、真实样本选取、二分/线性模式。已完成——原图的二分轨迹和标定时间是编的, 现在读日志。
fig_frontier	6.2	审计数据来源、调参、style.py 里硬编码的 RaBitQ 点。无上限阴影。阴影已删。
fig_alpha	6.3.1	当前的 ranking-loss 轴有效; 旧的「25×」和等召回图注需更正。已完成。
fig_calibration	6.3.2	保持 dataset+nprobe 键; 区分旧的分数容忍度证据、重复模式、平台阈值敏感性。已完成——面板 (c) 原本混了线性和二分两个算法。
fig_economics	6.3.3	已完成——四个未改变预算的配置单独归类, 不画在对数轴上。
fig_lutgroups	6.4.2	新增, 已完成——粒度扫描与 2×2 匹配消融。
fig_hierarchy	6.4.1	实测最高召回、匹配设置、无普适上限。
fig_ablation	6.4	分开新颖性层级并冻结最终匹配消融。已分组。
fig_negatives	6.5	受控复用代理; 实测事实 vs 推断机制。已完成。
fig_batch	6.6	固定 nprobe、两边召回; 核对实际查询数。已标注。
fig_build / fig_cost / fig_memory	6.7	冷构建来源、测量配置、模型对账、竞品实测值。已完成。

最终印刷尺寸下文字至少约 7 pt, 字体可嵌入可读, 标记可区分。图注与脚本注释必须与所绘的量一致。早先说「所有图的修正已完成」的评审, 不能取代对当前脚本的检查。

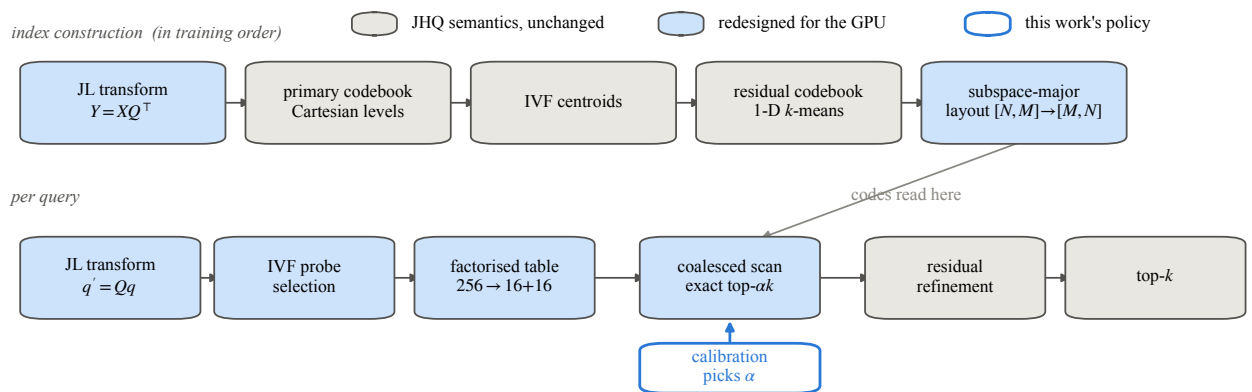
证据就绪度与剩余实验 TODO

主张	当前就绪度	定稿前需要
精确的笛卡尔表分解 alpha 变化幅度与删失端点	语义/代数完成, 实现前提在训练期检查 在测试网格内的冻结诊断召回证据完 成 (alpha6.log)	隔离计时消融已完成 (fig_lutgroups)。 说明 nprobe 与平台约定; 不用诊断 QPS。
标定经济性汇总	行级算术完成 (paper_fronts.log)	经济图已完成; 限定计时范围、质量与 平稳性。
最终策略普遍能找到足够 alpha	** 仍受阻 [TBD], 但已有留出验证 **	2000 次重采样表明 S=32 不是普适拐 点; 还需最终槽位规则敏感性与删失检 查。
GPU 工作区域比较	冻结曲线存在, 协议待补全	完整基线网格、训练/缓存谱系、插值 来源、等召回表。
比原版 CPU JHQ 更快	** 仍受阻 [TBD] **	可复现的最强 CPU 配置、明确来源、 匹配训练/召回、pinned 计时。
残差层级的价值	冻结消融存在	审计匹配性, 避免把扫描最大值称作上 限。
显存权衡 固定 nprobe 的批敏感性	JHQ 与四个 RaBitQ 数据集有实测总量 冻结测量存在	对齐工作区配置与建模/实测对账。 核实批的构造; 等召回的批结论仍受阻。
跨 k、漂移工作负载、跨 GPU 的推广	** 未测量 [TBD] **	要么做相关研究, 要么明确保留为局限。
Vogue 不平衡与可重复性数字	笔记里有记录, 原始来源不完整	发表数字前先冻结直方图与重复运行 证据。

优先级: 补齐必需的 CPU 基线与基线协议; 验证最终 alpha 策略与质量例外; 冻结匹配的表示/执行消融与构建/测量来源; 解决批计数、可重复性与诊断证据。更大的 k、更大的独立批、漂移和第二块 GPU 若未测量, 可以明确保留为范围局限。绝不为了让某个叙述成立而改动实验数据或实现。

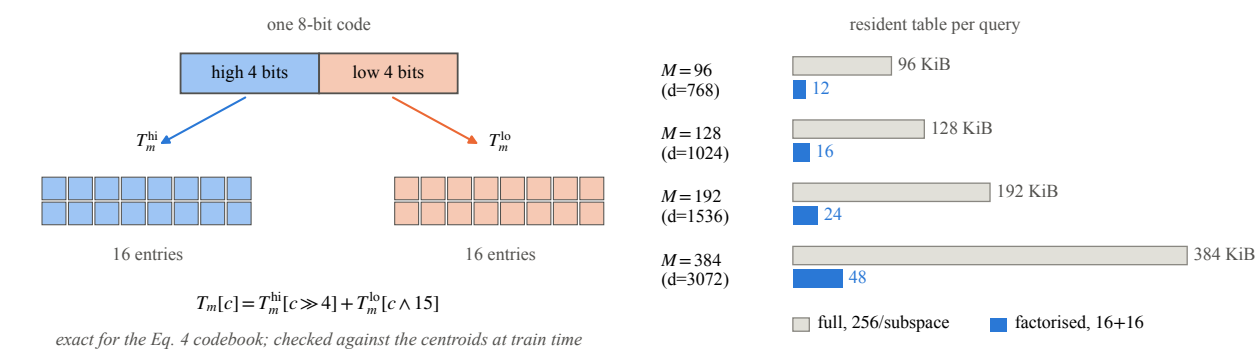
第二部分——图鉴

每张图整页宽, 底下是它的中文说明: 测的是什么、关键数字、以及写图注时不能说错的地方。英文原始 docstring 见 `handbook.pdf` 的第二部分。

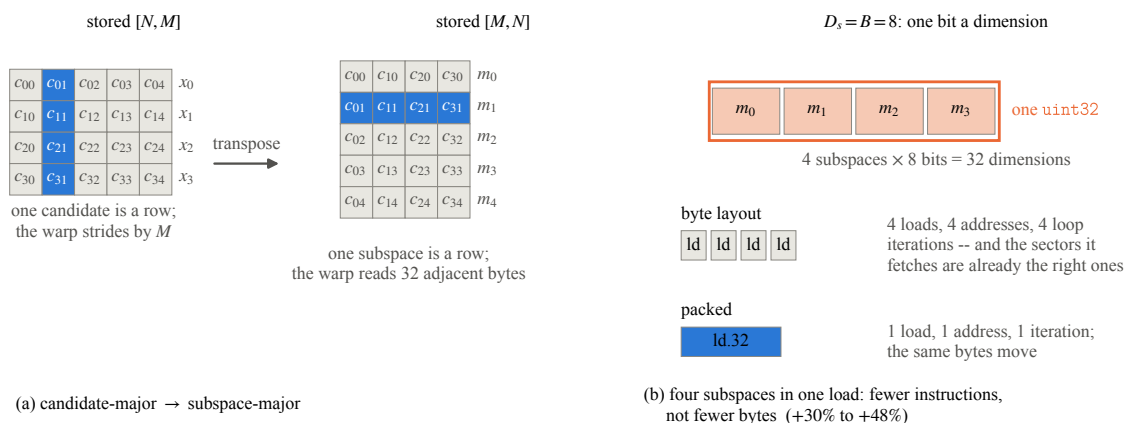
fig_pipeline the JHQ-GPU pipeline, shaded by what changed

JHQ-GPU 的流水线示意, 按「哪些是 GPU 重设计、哪些是原样保留的 JHQ 语义」着色。构建行按代码真实的训练顺序画 (JHQ_TRAIN_PHASE): JL 变换 $\rightarrow$ 主码本 $\rightarrow$ 主码编码 $\rightarrow$ **IVF** 质心 $\rightarrow$ 残差码本。残差码本在 IVF 质心之后, 原图画反了。残差码本用采样训练, 不等全部向量分配完毕, 所以画成反过来会暗示一个不存在的依赖。记号用 Q (不是 Π), 图里不写公式编号。

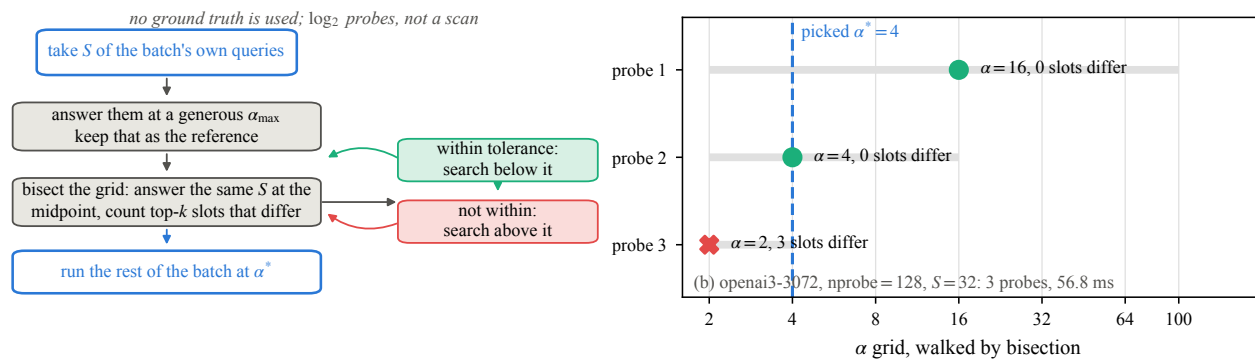
fig_lut the Cartesian factorisation, and what it saves per query



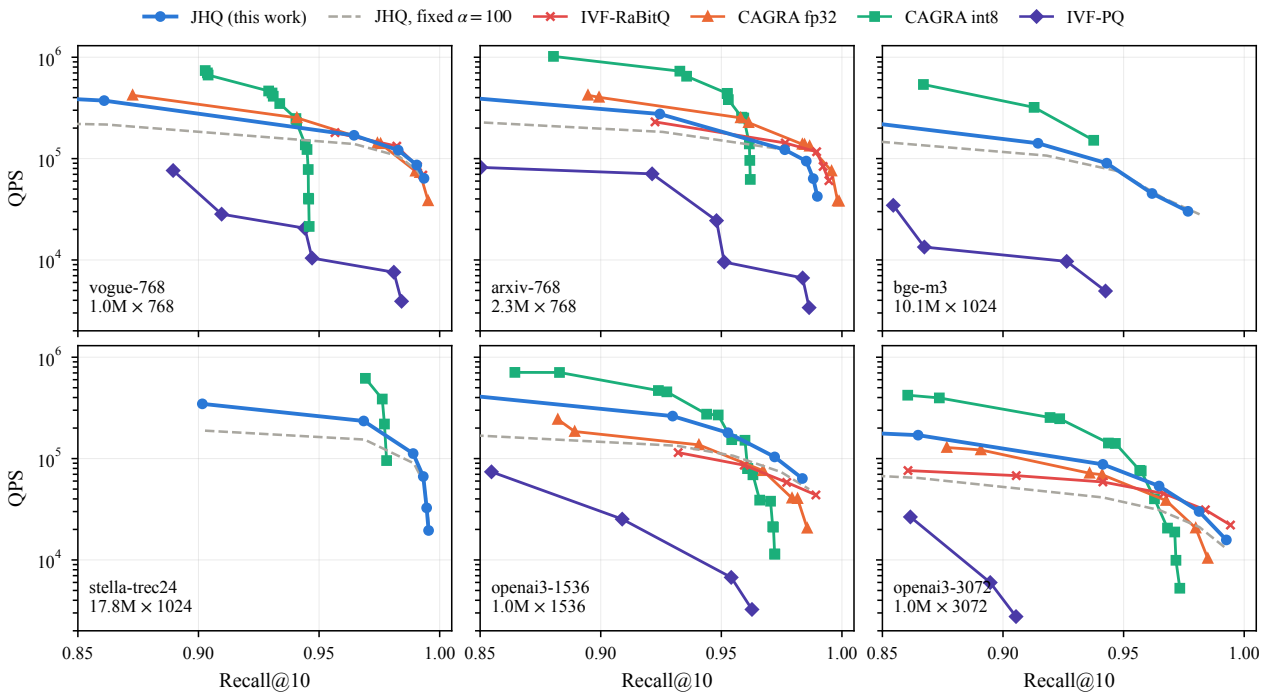
笛卡尔分解: 每子空间 256 项 $\rightarrow$ 16+16 = 32 项, 以及每查询省下的表构建工作。这些计数是代数推论, 不是加速比。分解精确成立的前提是主码本本身就是一维层级的笛卡尔积; 实现里 `verify_split_lut()` 在训练期对着质心逐组检查,Lloyd 细化过的码本会抛异常而不是静默算错。写图注时要提一句代价: 每子空间由一次查表变成两次, 别让读者把八倍的存储缩减读成八倍加速。

fig_layout the transpose, and four subspaces in one load

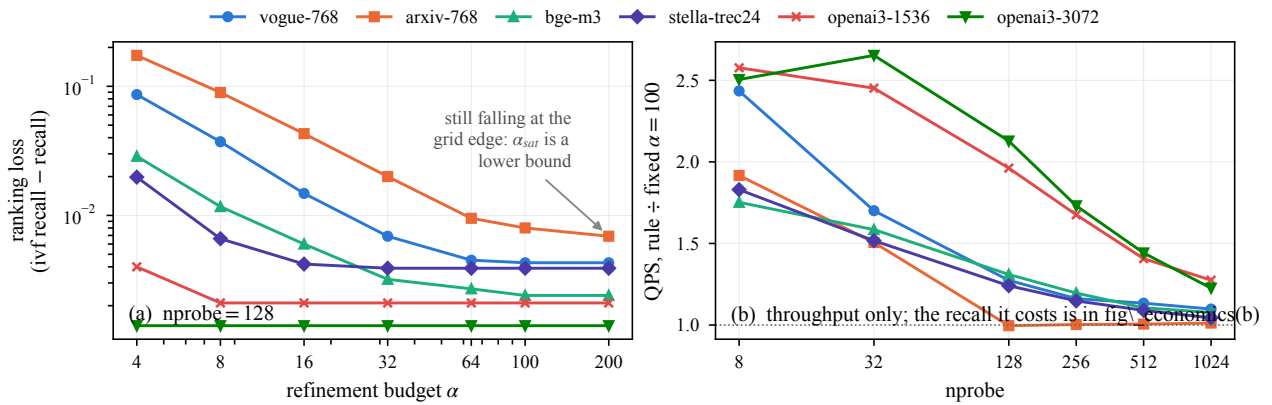
两件事, 必须分开讲。(a) 转置 ($[N, M] \rightarrow [M, N]$) 是合并访存的修复: 32 个线程原本按 M 步长跨读, 现在读 32 个连续字节。(b) 字打包 (四个子空间进一个 uint32) 不是合并访存的修复, 也不减少搬运的字节。Pascal 之后全局访问按 32 字节 sector 服务, (a) 之后取的本来就是它要用的那些 sector。+30%~+48% 来自指令数: 一条 load 换四条、一次寻址、循环迭代减到四分之一。绝不要重复「128 字节行浪费 75%」那个说法——那是错的, 而且曾经写进过这份项目的多处文档。

fig_rule how the budget is chosen, and one real calibration

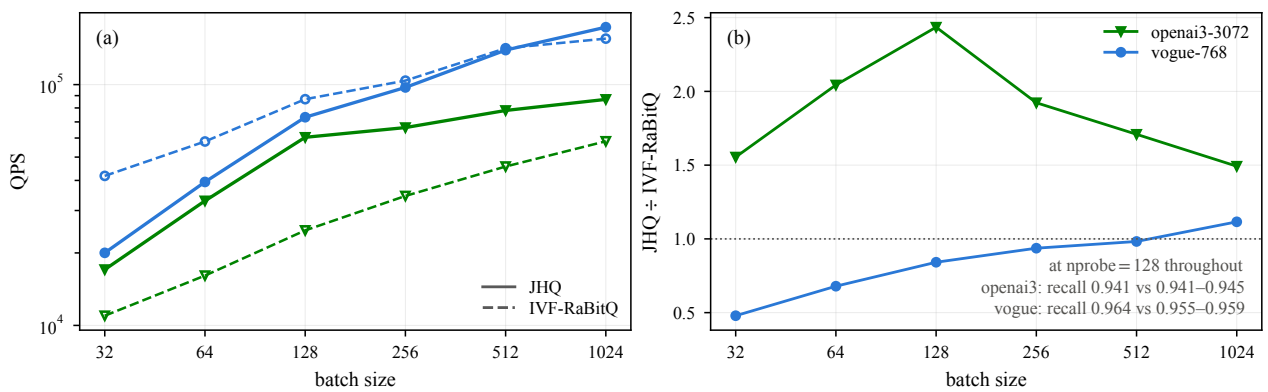
标定规则的机制, 加一次真实的标定过程。 ** 右图的数据从 `alpha_fast.log` 解析, 图里没有一个手写数字 ** (旧版把二分轨迹硬编码成 $32 \rightarrow 8 \rightarrow 4$ 全接受、标定时间写 8.9 ms; 它声称展示的那次运行实际是探 16、4、2, 耗时 56.8 ms)。规则走的是二分, 不是逐级下降。所以「停在第一次拒绝所以单边」是错的: *vogue* 上第一次探测就被拒, 搜索继续往上走 (16 拒绝 $\rightarrow$ 64 接受 $\rightarrow$ 32 拒绝 $\rightarrow$ 选 64)。二分成立需要「接受谓词随 α 单调», 而这一条可以论证: 精确 top-ck 选择给出嵌套候选集, 所以不一致槽位数随 α 单调不增。论文应写这个论证, 不要写单边性。

fig_frontier six-panel recall-QPS frontier

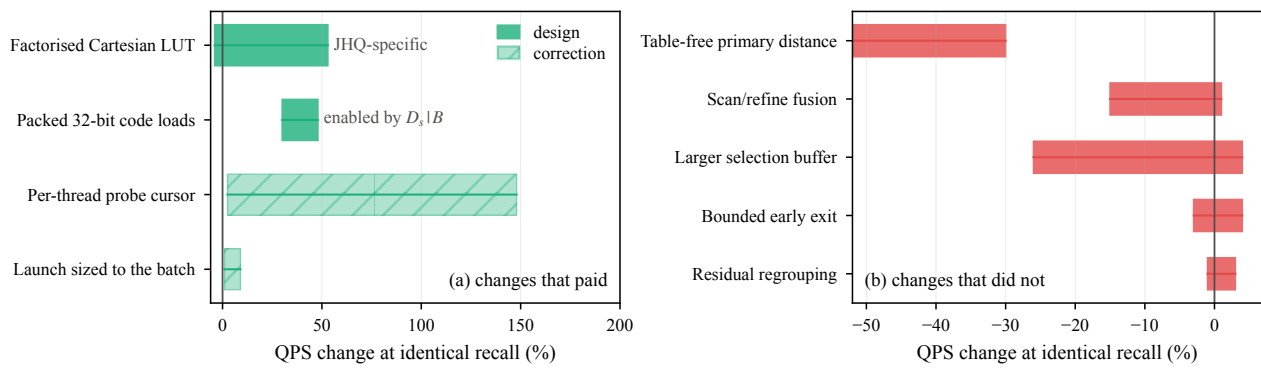
六数据集的 recall-QPS 前沿, 同一张卡上所有方法。虚线灰线是固定 $\alpha=100$ 的 JHQ, 到实线的距离就是标定规则的价值。没有任何区域被涂成「达不到」——曲线止于它自己扫描止住的地方, 基线最高点低于 JHQ 不等于它上不去, 只等于没被要求上去。只在曲线重叠处比较。唯一两处「做不到」的断言是显存分配失败 (CAGRA fp32 和 IVF-RaBitQ 在 stella / bge-m3), 它们用文字说明。

fig_alpha ranking loss vs alpha; what the rule is worth

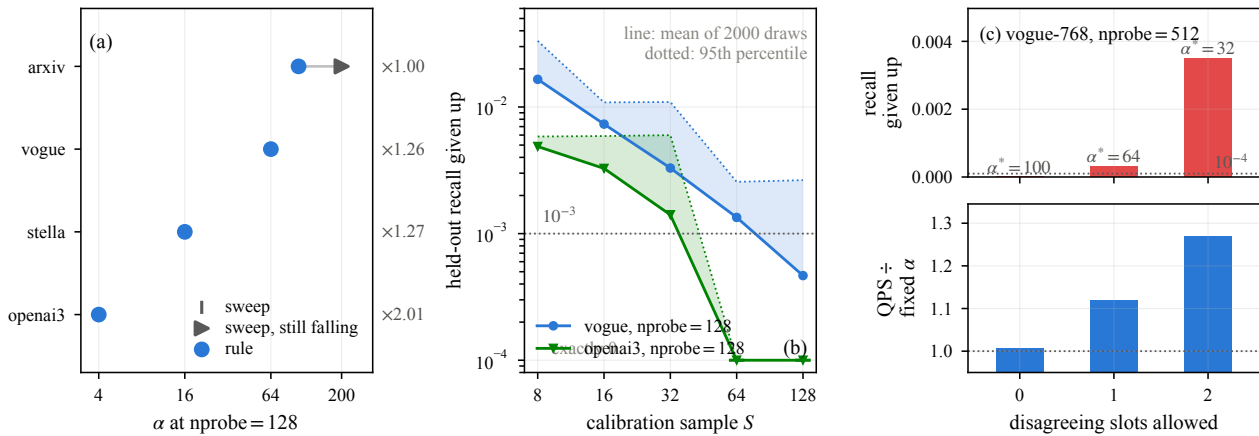
(a) 排序损失对 α , nprobe=128。纵轴是 **ivf_recall - recall**, 即「路由已经打开了那个桶、而主过滤和精化仍然没找回来」的那部分——路由损失已被剔除, 所以叫 **ranking loss** 才成立。用 $1-\text{recall}$ 会把路由损失也算进去, 而 α 够不到没被打开的桶。数据来自 `alpha6.log` (唯一记录了 `ivf_recall` 的那次运行, DIAG 构建; 它的 QPS 列不可用, 但召回类的量不受计时影响)。**arxiv-768** 在网格最大处仍在下降, 所以它的饱和点是下界, 图上有标注。**openai3-3072** 从 $\alpha=4$ 起就平了——这就是 $25\times$ 跨度的来源, 但不要写成「 $25\times$ 」, 写「4 到至少 200」。(b) 只是吞吐比; 它要的召回代价在 `fig_economics(b)`。

fig_batch throughput and the JHQ/RaBitQ ratio against batch

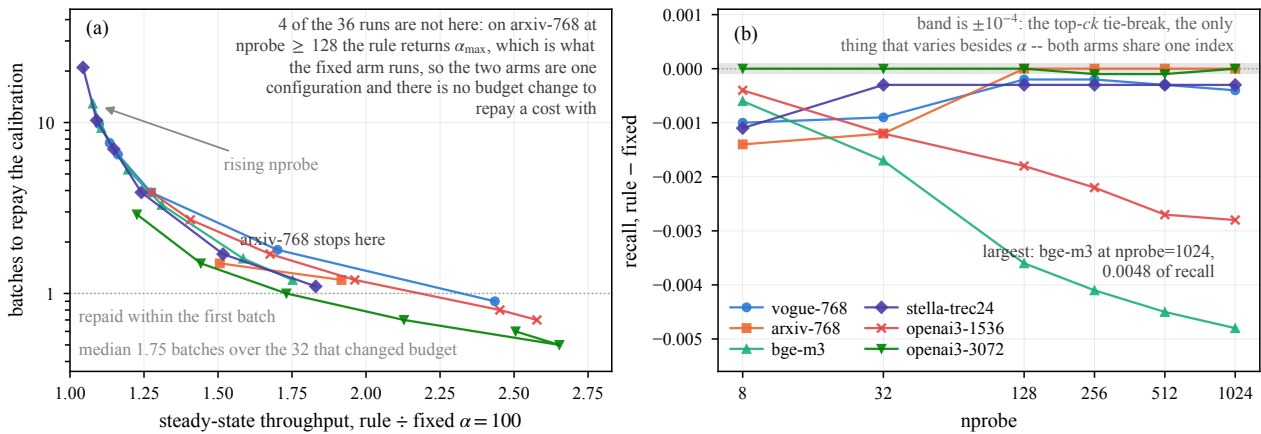
固定 $nprobe=128$ 下的批大小敏感性, 不是等召回比较。面板 (b) 写出了各自的召回: openai3-3072 是 0.9414 对 0.9405–0.9450(比值略偏向 JHQ), vogue-768 是 0.9645 对 0.9549–0.9592(JHQ 是在更高召回上被计时的)。比值随批大小在两个数据集上朝相反方向移动。不要靠复制约 1000 条真实查询造一个「独立的 10k 工作负载」——复制本身就能靠 L2 复用把吞吐抬高 4%~26%。

fig_ablation what paid, and what did not

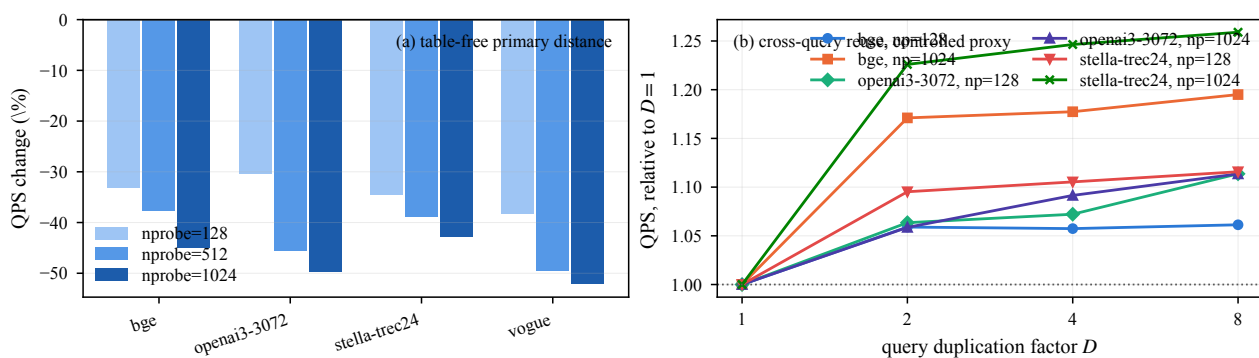
(a) 付了钱的改动, 按主张类型分三组: JHQ 专有 (分解 LUT)、由 $D_s | B$ 使能 (字打包)、修正 (游标、按批发射)。修正画成斜纹, 不必读者信图注。自适应 α 不在这张图上——它是策略结果, 增益是对固定 $\alpha=100$ 而非等召回, 而且带召回代价, 柱状图承载不了。它属于 §6.3。LUT 那根柱是 -4% 到 $+53\%$, 宽度本身就是发现 (见 fig_lutgroups), 不是噪声。误差区间是跨配置的范围, 不是置信区间; 不同行背后的运行不是同一组, 所以宽度可比而行与行不构成一个受控实验。(b) 没付钱的五个改动。

fig_calibration the rule against the sweep; sample size; tolerance

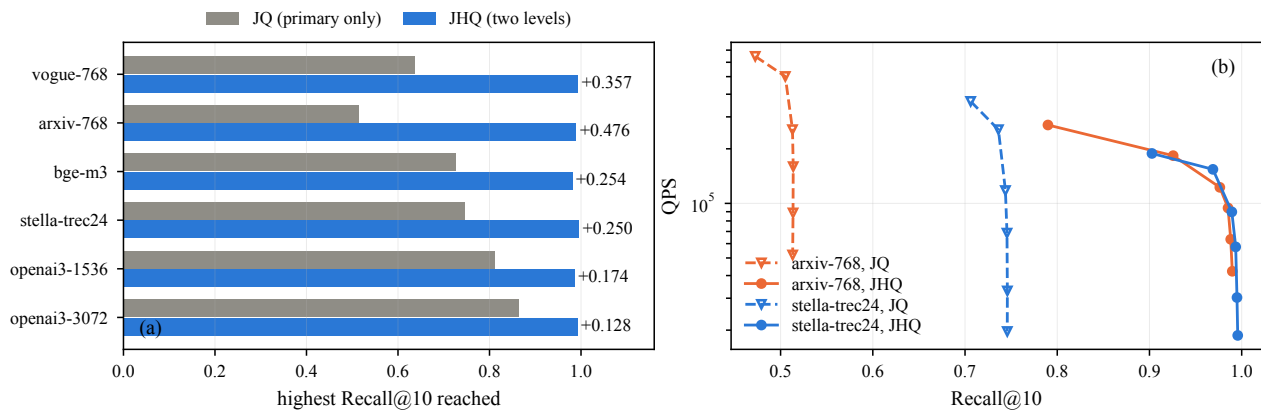
(a) 规则选的 α 对扫描找到的 α , 按 (dataset, nprobe) 键控——只画 nprobe=128, 因为那四个数据集在 nprobe=512 上没有跑过扫描, 复用另一个 nprobe 的答案等于假设掉要测的东西。三个精确命中; arxiv 不是规则出错, 是它的饱和点在规则自己的 $\alpha_{\max}=100$ 之上, 规则返回 $\alpha_{\max}$ 并正确报告没什么可赢。(b) 样本量, 2000 次重采样, 只在留出查询上打分。线是均值, 点线是 95 分位。「 $S=32$ 是拐点」不成立: vogue 在 $S=32$ 只有 27% 落在 $1e-3$ 内 (均值 0.0033、p95 0.0110), 要到 $S=128$ 才 89%; openai3-3072 在 $S=32$ 是 76%、 $S=64$ 是 98%。拐点是工作负载的性质。(c) 容忍度, 三个点全部来自二分 (旧版混进了线性变体的那一行, 三根柱子两个算法)。噪声参照是 $1e-4$ 不是 $1e-3$: 两条臂共用一个索引、一套码本, 唯一变的是 α , 残余只有 top-ck 的并列打破。

fig_economics what the calibration costs, and when it is repaid

标定的经济性——每个增益数字都以批次数为条件,这一点必须和增益一起报。(a) 增益对回本批次。形状本身是论点:probe 列表变长 $\rightarrow$ 鞍点上移 $\rightarrow$ 相对固定 $\alpha=100$ 的增益缩水,同时标定自己变贵 (2.0 $\rightarrow$ 47.5 ms), 同一个原因。所以成本最大的地方恰好是增益最小的地方。32 个真正改变了预算的配置: 增益全部 $> 1(1.044 - 2.653\times)$, 回本 0.5–21.0 批、中位数 1.75。四个配置不在图上:arxiv 在 nprobe ≥ 128 时规则返回 $\alpha_{\max}=100$, 而固定臂跑的也是 100 ——两条臂是同一个配置,gain 是重复计时的 $\pm 1.2\%$ 波动,batches_to_repay(179/108/58) 是拿标定成本除以噪声。那不是经济学结果,是 $1/\text{噪声}$ 。它们的召回差恰好是 0.0000, 是同一事实的另一面。注意: 增益与回本的反相关大半是定义性的 ($B^* = (T_{\text{cal}}/T_{\text{rule}})/(g-1)$, $g \rightarrow 1$ 时发散)。图的价值在它报出的量级, 不是一个机制发现。(b) 交易的另一半: 召回代价。带是 $\pm 1e-4$ 。不要说 **bge-m3** 是「唯一真实代价」——地板放到 $1e-4$ 之后,openai3-1536 的 -0.0028 同样是代价。报区间、点名最大的那个。

fig_negatives the table-free distance, and reuse as a controlled proxy

(a) 免表主距离 (v54): 代数上更省表, 实测慢 30%~52%。表的字节更少不等于执行更快。(b) 跨查询复用, 受控复用代理, 不是上界。复制查询让 D 条查询以完全相同的顺序探完全相同的桶, 比真实查询按粗分配聚类后的一致性还高; 增益在 $D=2$ 就饱和, 说明 96 MB 的 L2 已经吃掉了大部分复用。它是代理不是上界: 复制固定了调度只变查询共性, 而真正的重写还会改调度 (跨组摊销表构建、围绕桶而非查询重排扫描), 那些不在这条轴上。

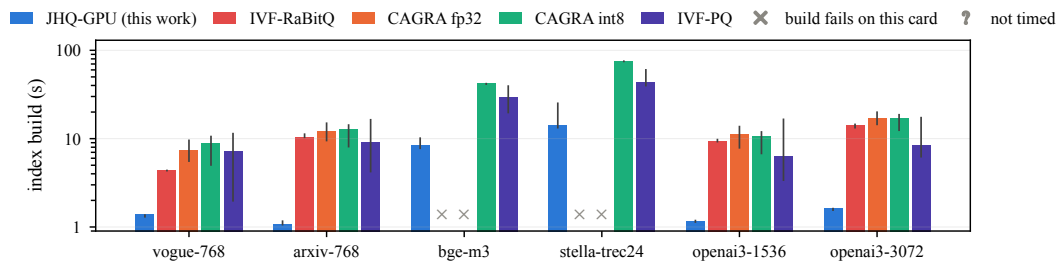
fig_hierarchy what the second level buys

第二级(残差)买到了什么。**(a)** 各数据集实测最高召回: 仅主码 vs 完整 JHQ, 差 +0.13 到 +0.48。**(b)** 两个数据集的完整前沿对比。把最大值称为实测最高召回, 不是架构上限。这个消融测的是层级, 不是一个调优完备的独立 JQ 系统。

fig_cost JHQ's build, split into training and encoding



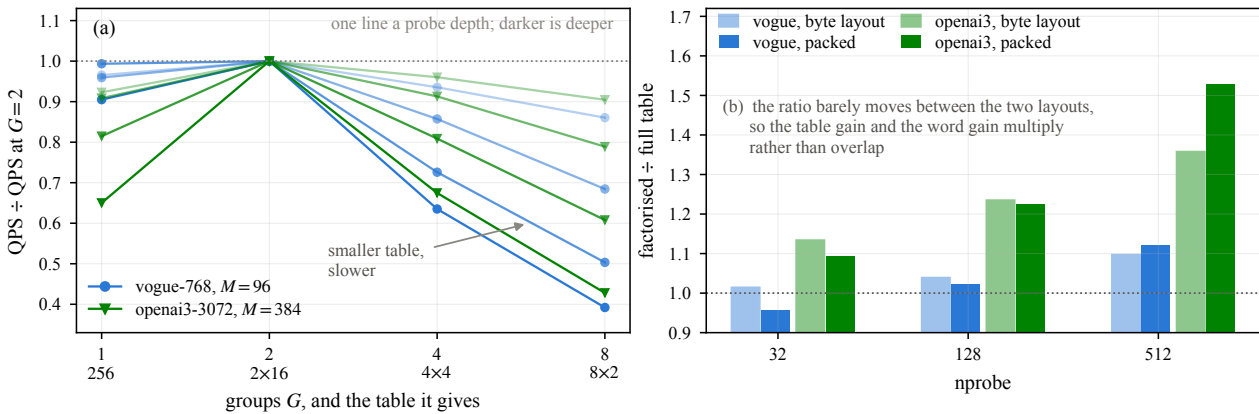
JHQ 自己的构建拆分: 训练 vs 编码。编码占大头且随 **N** 扩展, 训练随 **nlist** 扩展——stella 上 3.9 s 对 9.5 s。原来并排的显存面板已删除: 它只用一个数比 JHQ 和 RaBitQ, 而 `fig_memory` 用完整分解做同一比较。

fig_build index build time, all five methods

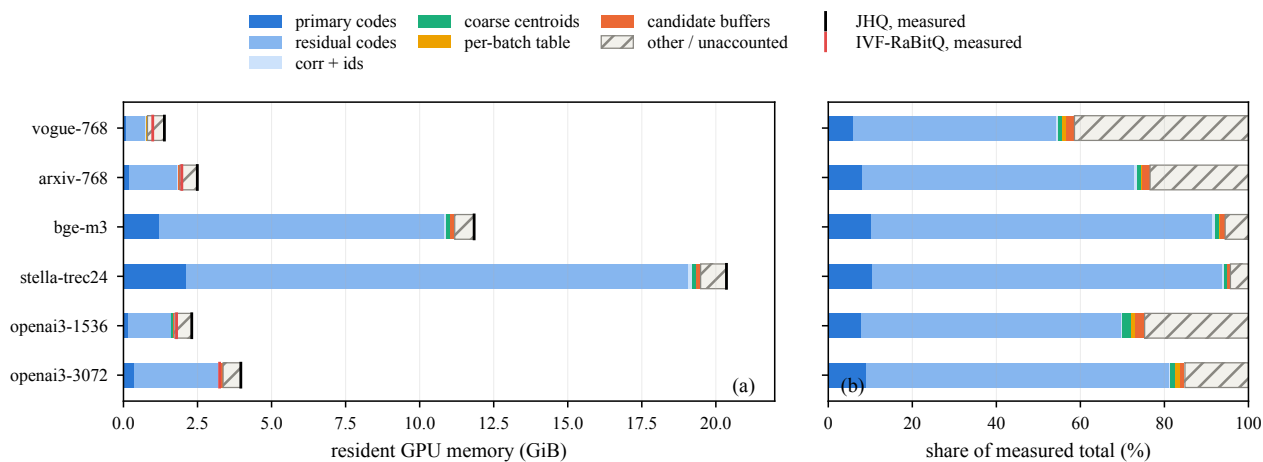
whisker: the range across swept configurations and repeat builds, not a confidence interval. JHQ's bar is its one cold training plus its median encode: training is cached after a dataset's first row, encoding is not and re-runs every row. All indexes train the coarse quantiser at 39 points a centroid.

跨方法的索引构建时间, 五个方法 × 六个数据集。数据本来就在, 只是 `train_ms` 列没进 `fronts.json`。JHQ 的柱是冷训练 + 中位编码。两个阶段缓存行为不同: `train` 有缓存 (只有 `nprobe=8` 那行是冷的), `add` 没有缓存且波动大 (stella 六次 9.1–21.8 s)。取 `max(train+add)` 会把「缓存过的训练 + 一次特别慢的编码」当成冷构建, 把 13.4 s 报成 22 s。空缺分两种, 是两个不同的断言: `×` = 这张卡上建不起来 (有日志为证); `?` = 我们没测过。把后者画成前者等于替基线断言一个没测过的限制。所有索引的粗量化器都按每质心 39 点训练。

fig_lutgroups why halves; and the table x layout 2x2



(a) 为什么是对半分。恒等式对任何数字划分都成立, 所以 8 个一位数字可切成 G 组: $G=1(256 \text{ 项})$ 、 $G=2(2 \times 16)$ 、 $G=4(4 \times 4)$ 、 $G=8(8 \times 2)$ 。四者召回在每个配置下完全一致 (恒等式要求如此, 这是先于计时的正确性检查)。 $G=2$ 在全部八个配置上都赢。输家说明了原因: $G=4$ 和 $G=8$ 的表更小 (16 项 vs 32 项) 却单调更慢, 比例正好跟每候选每子空间的加载数走 (2、4、8)。所以扫描不是受表大小限制的——表一旦小到能无冲突进共享内存, 再缩只白搭指令。这和免表距离、字打包是同一机制的第三个方向: kernel 是 issue-bound 的。(b) 2×2 : 完整表 vs 分解表 $\times$ byte 布局 vs packed 布局。分解的价值强烈依赖 M: $M=96$ 时 $-4\% \sim +12\%$, $M=384$ 时 $+23\% \sim +53\%$ 。机制在算术里——256 项表是 $M \times 256 \times 4$ 字节, $M=96$ 是 98 KiB (carveout 放得下), $M=384$ 是 393 KiB (放不下)。分解恰好在完整表放不下的地方才值钱, 这是扩展性质, 比平均值更有用。results/v47_split_lut/ 的「+6~14.5%、28 格无一为负」只对它测的那个 M 成立 (全部 $M=96/128$), $M=384$ 上低估了四倍, 而 $M=96$ 确实有一格为负。两笔收益近似独立: 分解在 byte 布局值 1.13/1.24/1.36, 在 packed 上值 1.10/1.23/1.53——相乘而非重叠, 这是「付两次红利」缺的证据。

fig_memory where JHQ\'s memory goes, and why it is above RaBitQ\'s

(a) 绝对显存分解, (b) 同一堆栈的占比。最后一段定义为 实测 - 建模, 所以柱子精确落在 `cudaMemGetInfo` 的总量上, 未解释的部分按真实大小画出来而不是被略去。它叫「**other / unaccounted**」而不是「**context + allocator**」——后两者是它预期的成因, 但这里没有任何东西单独测过它们。占比面板暴露一个发现: 未归因余量在 **vogue** 上占 **45%**、**stella** 上只占 **5%** ——固定开销被数据集规模摊薄。